

Создание диспетчера команд

Лабораторная работа 12

Было изучено содержание четырех лабораторных практикумов каждым участником команды и сделан выбор в сторону выполнения лабораторного практикума 14 и лабораторной работы в нем номер 12

Суть работы заключается в создании диспетчера команд – программного модуля, обеспечивающего диалог с пользователем и выполняющего координацию работы других модулей программы

Для разработки диспетчера команд была написана вспомогательная процедура Read_byte, выполняющая ввод с клавиатуры любого символа – обычного или управляющего. При этом код ASCII символа возвращается в регистре AL, а в регистре AH возвращается соответствующий скан-код BIOS.

Процедура Read_byte помещена в начало файла Kbd_io.asm

```
org 100h
;      Считывает код символа с клавиатуры
; -----
; Выход: AL - код ASCII символа (0 - управляющий символ)
;      AH - скан-код символа
;
Read_byte:
    mov ah,0 ; Ввод символа
    int 16h ; без эха
    ret
;
```

Для проверки был добавлен отладочный код

```
org 100h
;      Считывает код символа с клавиатуры
; -----
; Выход: AL - код ASCII символа (0 - управляющий символ)
;      AH - скан-код символа
;
Read_byte:
    mov ah,0 ; Ввод символа
    int 16h ; без эха
    mov dx, ax
    call Write_word_hex
    ret
%include 'Video_io.asm'
;
```

Проверка работы процедуры

Был осуществлен ввод сначала символа «А», а затем управляющего символа F5 со скан-кодом 3Fh

```
C:\PROGRA~1\NASM>nasm Kbd_io.asm -o Kbd_io.com
C:\PROGRA~1\NASM>Kbd_io.com
1E41
C:\PROGRA~1\NASM>Kbd_io.com
3F00
```

Далее написана главная подпрограмма (процедура) Dispatcher, выполняющей совместно с рассматриваемой далее процедурой Command (центральной особенностью ее алгоритма является использование таблицы переходов) функции диспетчера команд.

Обе процедуры, а также таблица переходов Table помещены в один и тот же файл Dispatch.asm

Проверка работы процедуры Dispatcher (для отладки в процедуре Command стоит заглушка и при вызове выводится: «Call command!»)

```
org 100h
jmp Dispatcher
; Данные программы
Sector db 0Dh, 0Ah, 'Enter command: $'
Sector2 db 0Dh, 0Ah, 'Call command!$'
;
; Координирует выполнение модулей редактора
; -----
;
Dispatcher:
    call Clear_screen          ; Очистка экрана
.back: mov ah, 09h            ; Код для вывода строки символов
    mov dx, Sector             ; Адрес строки для вывода
    int 21h                   ; Вывод строки
    call Send_crlf             ; Перевод строки
    call Read_byte             ; Ввод символа с клавиатуры для получения кода
    cmp al, 0                  ; Проверка управляющий символ или нет
    jz .cmp_f10                ; Если управляющий (al = 0), то переход к сравнению с f10
    cld                         ; Если не управляющий, то в cf помещаем ноль
    jmp .check                 ; Переход к проверке флага cf
.cmp_f10: cmp ah, 44h          ; Проверка лежит ли в символе код f10
    je .exit                   ; Если да, то завершение программы
    call Command               ; Вызов процедуры для выполнения команды
    cld                         ; Запись в cf нуля
    jmp .check                 ; Переход для проверки cf
.exit: stc                     ; Запись в cf единицы
.check: jnc .back              ; Проверка флага cf, при cf = 0 осуществляется переход
    int 20h                    ; Возвращение управления
%include 'Kbd_io.asm'           ; Подсоединение процедур
%include 'cursor.asm'           ; ---//---
;
```

Был осуществлен ввод символа «А», а затем управляющих символов F5, F9, F10 со скан-кодами 3Fh, 43h, 44h соответственно. Без отладочных выводов программа просто ждет ввода управляющего символа и при вводе символа отличного от управляющего экран очищается и ввод повторяется. Таким образом для пользователя действие происходит только при нажатии управляющего символа (при нажатии F10 происходит завершение программы)

```

Enter command:
1E41
Enter command:
3F00
Call command!
Enter command:
4300
Call command!
Enter command:
4400
C:\PROGRA~1\NASM>

```

Процедура Command

Данная процедура получает в регистре DL код ASCII, соответствующий команде, и в зависимости от этого кода вызывает процедуру, выполняющую заданное командой действие

```

; Антипов Арсений, Красников Даниил, Красникова Диана, Угарин Никита
; Интерпретатор команд
; -----
;
; Входы: DL - скан-код, соответствующий команде
; Чтение: Table - таблица переходов
; Антипов Арсений, Красников Даниил, Красникова Диана, Угарин Никита
Command:
    push bx
    mov bx, Table          ; BX <- адрес таблицы
.M1: cmp byte[bx], 0ffh    ; Конец таблицы?
    je .Exit              ; Да, кода нет в таблице
    cmp dl, [bx]          ; Это вход в таблицу?
    je .Dispatch          ; Да, выполнить команду
    add bx, 3             ; Нет, переход к
    cld                  ; следующему
    jmp .M2               ; элементу таблицы
.Dispatch: inc bx         ; Адрес процедуры
    call [bx]             ; Вызов процедуры
.Exit: stc               ; CF <- 1
.M2: jnc .M1             ; Повторение для нового элемента таблицы
    pop bx
    ret
; Антипов Арсений, Красников Даниил, Красникова Диана, Угарин Никита
;

```

Таблица переходов к конкретной функции, в зависимости от нажатой клавиши F

```

;
; Таблица содержит скан-коды управляющих клавиш и адреса процедур,
; выполняющих соответствующие команды
;
Table db 3bh          ; <F1>
      dw Init_sector
      db 3ch          ; <F2>
      dw Write_sector
      db 3dh          ; <F3>
      dw Next_sector
      db 3fh          ; <F5>
      dw Prev_sector
      db 40h          ; <F6>
      dw N_sector
      db 0ffh         ; Конец таблицы

```

Работа программы с отладочным выводом (можно видеть какие клавиши были нажаты и какие действия им соответствуют, при нажатии клавиши отличной от управляющего символа ввод необходимо повторить, для выхода надо нажать F10)

```

Enter command:
1E41
Enter command:
3B00
Display the initial sector
Enter command:
3C00
Write the corrected sector to memory
Enter command:
3D00
Display the next sector
Enter command:
3F00
Display the previous sector
Enter command:
4000
Display sector N, where 0 <= N <= 256
Enter command:
4400
C:\PROGRA~1\NASM>

```

Полный код программы, реализующей диспетчер команд

```

org 100h
jmp Dispatcher
; Данные программы
Sector db 0Dh, 0Ah, 'Enter command: $'
;
; Координирует выполнение модулей редактора
; -----
;
Dispatcher:
    call Clear_screen      ; Очистка экрана
.back: mov ah, 09h         ; Код для вывода строки символов
    mov dx, Sector         ; Адрес строки для вывода
    int 21h                ; Вывод строки
    call Send_crif         ; Перевод строки
    call Read_byte         ; Ввод символа с клавиатуры для получения кода
    cmp al, 0              ; Проверка управляющий символ или нет
    jz .cmp_f10            ; Если управляющий (al = 0), то переход к сравнению с f10
    cld                    ; Если не управляющий, то в cf помещаем ноль
    jmp .check             ; Переход к проверке флага cf
.cmp_f10: cmp ah, 44h      ; Проверка лежит ли в символе код f10
    je .exit              ; Если да, то завершение программы
    mov dl, ah             ; Запись вводимой функции
    call Command           ; Вызов процедуры для выполнения команды
    cld                    ; Запись в cf нуля
    jmp .check            ; Переход для проверки cf
.exit: stc                 ; Запись в cf единицы
.check: jnc .back         ; Проверка флага cf, при cf = 0 осуществляется переход
int 20h                   ; Возвращение управления
%include 'Kbd_io.asm'      ; Подсоединение процедур
%include 'Cursor.asm'     ; -----
%include 'stch.asm'       ; -----
;

```

```

; Антипов Арсений, Красников Даниил, Красникова Диана, Угарин Никита
; Интерпретатор команд
; -----
;
; Входы: DL - скан-код, соответствующий команде
; Чтение: Table - таблица переходов
;
Command:
    push bx
    mov bx, Table ; BX <- адрес таблицы
.M1: cmp byte[bx], 0ffh ; Конец таблицы?
    je .Exit ; Да, кода нет в таблице
    cmp dl, [bx] ; Это вход в таблицу?
    je .Dispatch ; Да, выполнить команду
    add bx, 3 ; Нет, переход к
    cld ; следующему
    jmp .M2 ; элементу таблицы
.Dispatch: inc bx ; Адрес процедуры
    call [bx] ; Вызов процедуры
.Exit: stc ; CF <- 1
.M2: jnc .M1 ; Повторение для нового элемента таблицы
    pop bx
    ret

; Антипов Арсений, Красников Даниил, Красникова Диана, Угарин Никита
; Таблица содержит скан-коды управляющих клавиш и адреса процедур,
; выполняющих соответствующие команды
;
Table db 3bh ; <F1>
    dw Init_sector
    db 3ch ; <F2>
    dw Write_sector
    db 3dh ; <F3>
    dw Next_sector
    db 3fh ; <F5>
    dw Prev_sector
    db 40h ; <F6>
    dw N_sector
    db 0ffh ; Конец таблицы

```